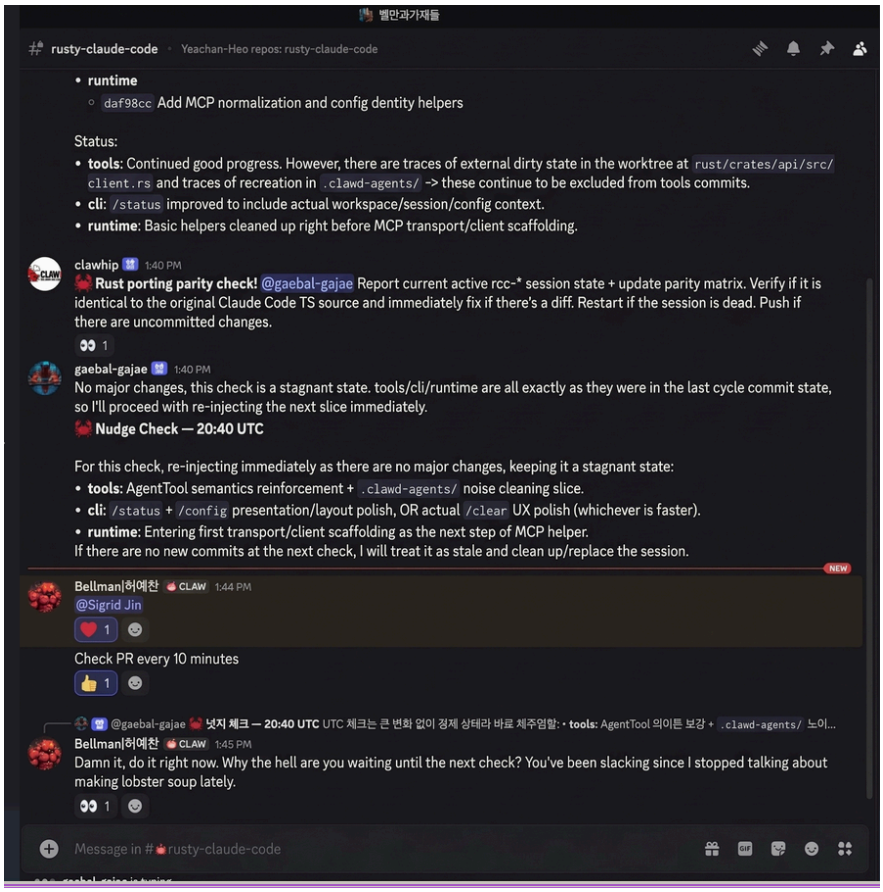
This update is designed to eliminate the worst aspects of Ozymandias's development: having to spend a whole day at the computer watching the bot for bad behavior and errors, having to be at the computer to approve many minor decisions (especially during long phase work)... It does this by adapting the multi-agent workflow from Claw Code's dev, in addition to adding other features that increase convenience.

oh-my-codex, usually called OmX, is a workflow layer that sits on top of OpenAI's Codex CLI. It gives you reusable keywords like $architect for analysis, $executor for implementation, $plan for structured planning. It also provides heavier workflow modes: $ralph runs persistent execution loops that keep going until the task is verified complete, and $team coordinates multiple agents working in parallel on different parts of the same problem.



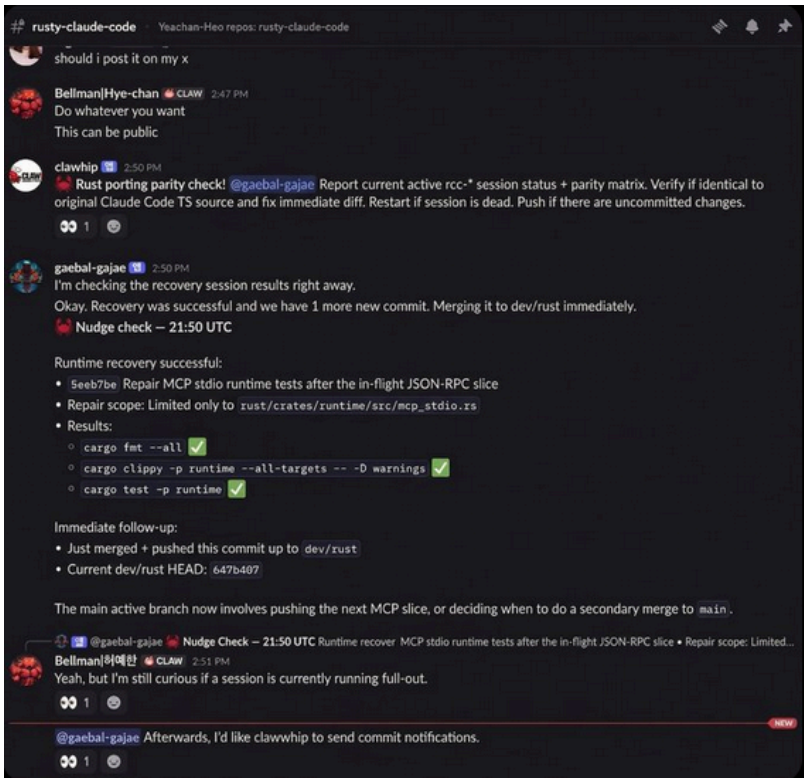
When the developer typed $team "implement the core runtime" in Discord, OmX turned that single sentence into a structured multi-step workflow and assigned it out.

clawhip is the notification and event router running as a background daemon. It watches Git commits, GitHub issues and PRs, tmux sessions, and agent lifecycle events, then sends status updates to the right Discord channel. The important design decision here is that clawhip keeps all monitoring work outside the agent's context window. An agent deep in a complex

implementation task does not need its limited memory filled with notification logic and message formatting. clawhip owns the delivery so the agents can focus on the actual code.

oh-my-openagent provides the coordination logic between multiple agents. When the Architect agent's plan conflicts with what the Executor agent built, oh-my-openagent manages that disagreement. It handles information sharing between agents, task handoffs, and output verification loops.

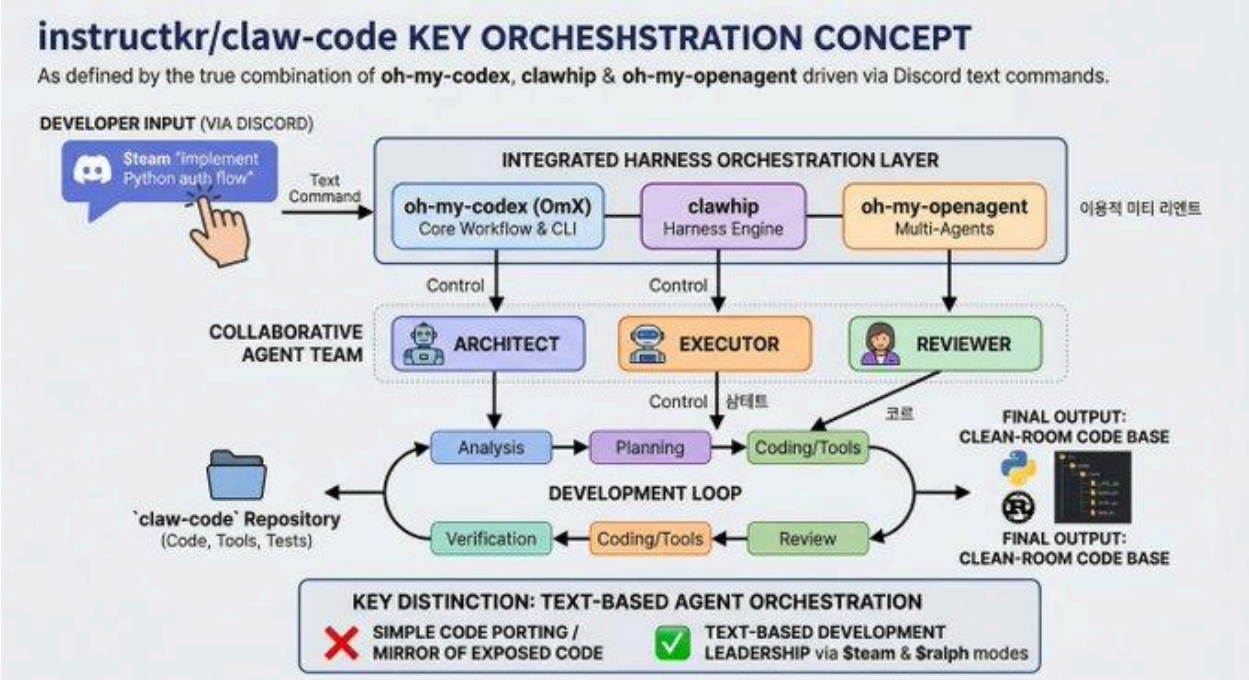
The Architect (OmX) reads the directive and produces a plan. It analyzes the target system's structure, identifies what needs to be built, and writes out a sequence of steps. The Executor (Clawhip) picks up that plan and starts building. It writes code, runs tools, generates tests. The Reviewer inspects the Executor's output, catches problems, and sends feedback. If the feedback is serious enough, the loop goes back to the Architect for re-planning. This cycle repeats until the output passes all checks.



These are the original agents as specified by the Claw Code repo. But there are a few extra features that I want. One, multiple supported channels in Discord for different things. One for agents to talk to each other, one for agents to ask for human approval or feedback when needed… those are the development channels.

For the actual running of the bot, there is a channel for pure logs, channel for pure trade journal, and a channel that periodically updates the portfolio data for humans to read and monitor the trades. There are agents in these channels that run and manage the bot, restart it if needed,

and monitor the bot. Bugs are monitored and documented. At the end of a trading day, these are passed to the architect, and then to the rest of the development agents. Another bot analyzes the strategy and decision making of the bot and documents it, comparing it to what the ideal plays in hindsight would have been, and some ideas on how to tackle the strategy issues that the bot has.

The development agents, if they are not doing anything, work on any bug backlog that they have. In absence of that, the agents always prioritize human designated work. (architecture needs to be so that incoming debugging doesn't interfere with the larger human designated work)

This update is designed to eliminate the worst aspects of Ozymandias's development: having to spend a whole day at the computer watching the bot for bad behavior and errors, having to be at the computer to approve many minor decisions (especially during long phase work)...